



Rust dev and build environments on Nix/NixOS

Managing a Rust toolchain with Nix and Flakes

Simon Shine <simon@simonshine.dk>

October 31, 2024



Motivation for talk

What

- ⚙ Use Nix to build Rust programs
- ⚙ Address problems of increasing complexity
- ⚙ Reason about effort vs. payoff for each problem / solution



Motivation for talk

What

- ⚙ Use Nix to build Rust programs
- ⚙ Address problems of increasing complexity
- ⚙ Reason about effort vs. payoff for each problem / solution

Why (not just stick to rustup and cargo?)

- ⚙ Manage external dependencies and additional build tools
- ⚙ Build projects across new machines and across time
- ⚙ Build projects in a way that is reproducible and declarative
- ⚙ Reuse the developers' environment in CI without Devcontainers
- ⚙ Produce build artifacts (e.g. executables) that can be deployed declaratively
- ⚙ Faster onboarding; have less “Works On My Machine `~\_(\_)\_/`”



Motivation for talk

What

- ⚙ Use Nix to build Rust programs
- ⚙ Address problems of increasing complexity
- ⚙ Reason about effort vs. payoff for each problem / solution

Why (not just stick to rustup and cargo?)

- ⚙ Manage external dependencies and additional build tools
- ⚙ Build projects across new machines and across time
- ⚙ Build projects in a way that is reproducible and declarative
- ⚙ Reuse the developers' environment in CI without Devcontainers
- ⚙ Produce build artifacts (e.g. executables) that can be deployed declaratively
- ⚙ Faster onboarding; have less “Works On My Machine `~\_(\_)\_/`”

How

- ⚙ Show ways to install and use a Rust toolchain on a Nix system
- ⚙ Some ways are hard to find, but don't complicate things (“down payment”)
- ⚙ Some ways complicate things (“upkeep”)

Conclusion

By the end of this talk, I hope to have convinced you that...

- ❄ ... a small amount of Nix will go very far
- ❄ ... a moderate amount of Nix solves real problems
- ❄ ... a large amount of Nix is a machine that feeds itself

Conclusion

By the end of this talk, I hope to have convinced you that...

- ❄ ... a small amount of Nix will go very far
- ❄ ... a moderate amount of Nix solves real problems
- ❄ ... a large amount of Nix is a machine that feeds itself

... but the machine is reliable, reproducible, and declarative!



Advertisement time!

November 5, 2024 (Tuesday):

Rust Hack Night #10: Rust <3 Nix

⚙️ **What:** Workshop-style, make Nix flakes for your Rust projects

⚙️ **Where:** KEA @ Guldbergsgade 29N, 2200 Copenhagen N

(check the Copenhagen Rust Meetup page)

December 5, 2024 (Thursday):

NixOS Install Party

⚙️ **What:** Join the federation, share and improve your configuration

⚙️ **Where:** DIKU @ Universitetsparken 1, 2200 Copenhagen N

(check the Greater Copenhagen NixOS Meetup page)

A quick poll...



How many of you...

... run NixOS?

... run Nix on another Linux or MacOS?

... have made a file ending with `.nix` for a project?

Grug Brained Developer: How do I install Rust?



How do I install rust?

■ Help



smn

Jun '20

Me want install rust.

Me search forum. Find bunch of random things.

Me go to rust-lang.org ⁸² copy command. No worky.

Me click “Other Installation Methods” on rust-lang.org ⁸². Me Ctrl-F “nix” find much Unix. Search stupid.

Me look manual: `nix-env -qa` for search package. Me `nix-env -qa rust`. No findy. Me `nix-env -qa '*rust*'`. Finger ouch. Invalid regex. Me `nix-env -qa '.*rust.*'`. Finger more ouchy. Find rustup but search stupid. Me sad.

Me want `nix-env -i rustup`. [@jonringer](#) say [no use nix-env](#). ⁵⁰.

Grug Brained Developer: How do I install Rust?



How do I install rust?

■ Help



Me look manual. Manual long. Me browse a bit until too sad. Me look configuration.nix. Me find

```
# List packages installed in system profile. To search, run:
# $ nix search wget
environment.systemPackages = with pkgs; [
  wget vim
];
```

Me `nix search rust`. Long list. Me find `nixpkgs.rustup`. Me add `rustup` after `vim`. Me `sudo nixos-rebuild switch`. Me afraid Xorg will go away again. Sometimes `sudo nixos-rebuild switch` make Xorg go away. Me anxiety now.

Xorg no go away. Me relieved.

Me `rustup toolchain install nightly`. Me less sad. But much late now.

Me `cargo new foo`. Have new project. Me `cargo run` in project. `error: linker cc not found`. Me more sad again. Me sleep now.

NixOS too much sad.

Overview

Install a system-global Rust toolchain on NixOS

TT / (° _ ° /)

❄️ Install the rustup package from nixpkgs

Overview

Install a system-global Rust toolchain on NixOS

- ❄️ Install the rustup package from nixpkgs

Install a project-specific Rust toolchain

- ❄️ Specify whether to run stable or nightly per project
- ❄️ Specify what additional build tools should be in my \$PATH
- ❄️ Cross-compilation (PC, embedded, wasm)
- ❄️ Provide a customized dev shell
- ❄️ Put Nix in my CI pipeline

Overview

Install a system-global Rust toolchain on NixOS

- ❄ Install the rustup package from nixpkgs

Install a project-specific Rust toolchain

- ❄ Specify whether to run stable or nightly per project
- ❄ Specify what additional build tools should be in my \$PATH
- ❄ Cross-compilation (PC, embedded, wasm)
- ❄ Provide a customized dev shell
- ❄ Put Nix in my CI pipeline

Blow the novelty budget and see what's out there

- ❄ Manage crates and workspaces using Nix flakes
- ❄ Use Nix flakes to maintain dependencies between repositories
- ❄ Re-establish caching (similar to how one might fight Docker's cache)
- ❄ Declaratively manage my VSCodium and its Rust plugins

Install a system-global Rust toolchain on NixOS

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
  ];  
}
```

Install a system-global Rust toolchain on NixOS

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
  ];  
}
```

command-line

```
$ sudo nixos-rebuild switch  
$ rustup default stable  
$ cargo init hello-world  
$ cd hello-world  
$ cargo run  
...  
Hello, world!
```

Install a system-global Rust toolchain on NixOS

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
  ];  
}
```

command-line

```
$ sudo nixos-rebuild switch  
$ rustup default stable  
$ cargo init hello-world  
$ cd hello-world  
$ cargo run  
...  
Hello, world!
```

Pros and cons

- ⚙️ That's it; you can go back to coding Rust now!
- ⚙️ Actually provides: cargo rustc rust-analyzer rustdoc rustfmt rustup ...
- ⚙️ Switch between stable / nightly, multiple targets, etc. the usual way
- ⚙️ Need to repeat rustup commands on new systems: Projects don't just build
- ⚙️ Need to switch toolchain between projects, but we have [rust-toolchain.toml](#)



Let's make it more realistic...

command-line

```
$ cargo add tokio --features=full  
$ cargo add request --features=json
```



Let's make it more realistic...

command-line

```
$ cargo add tokio --features=full  
$ cargo add reqwest --features=json
```

src/main.rs

```
#[tokio::main]  
async fn main() -> Result<(), Box<dyn std::error::Error>> {  
    let response = reqwest::get("https://ipinfo.io/")  
        .await?  
        .json::<std::collections::HashMap<String, String>>()  
        .await?;  
    println!("{}", response);  
    Ok(())  
}
```



Let's make it more realistic...

```
$ cargo build
```

Let's make it more realistic...

```
$ cargo build
```

```
...
```

```
error: failed to run custom build command for `openssl-sys v0.9.103`
```

Let's make it more realistic...

```
$ cargo build
```

```
...
```

```
error: failed to run custom build command for `openssl-sys v0.9.103`
```

```
...
```

```
The pkg-config command could not be found.
```

Most likely, you need to install a pkg-config package for your OS. Try ``apt install pkg-config``, or ``yum install pkg-config``, or ``pkg install pkg-config``, or ``apk add pkgconfig`` depending on your distribution.



Let's make it more realistic...

```
$ cargo build
```

```
...
```

```
error: failed to run custom build command for `openssl-sys v0.9.103`
```

```
...
```

```
The pkg-config command could not be found.
```

Most likely, you need to install a pkg-config package for your OS. Try ``apt install pkg-config``, or ``yum install pkg-config``, or ``pkg install pkg-config``, or ``apk add pkgconfig`` depending on your distribution.

```
...
```

Could not find directory of OpenSSL installation, and this ``-sys`` crate cannot proceed without this knowledge. If OpenSSL is installed and this crate had trouble finding it, you can set the ``OPENSSL_DIR`` environment variable for the compilation process.

Make sure you also have the development packages of openssl installed. For example, ``libssl-dev`` on Ubuntu or ``openssl-devel`` on Fedora.

```
...
```



Let's make it more realistic...

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
    pkgs.pkg-config  
    pkgs.openssl  
  ];  
}
```



Let's make it more realistic...

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
    pkgs.pkg-config  
    pkgs.openssl  
  ];  
  environment.sessionVariables = {  
    PKG_CONFIG_PATH = "${pkgs.openssl.dev}/lib/pkgconfig";  
    LD_LIBRARY_PATH = pkgs.lib.makeLibraryPath [  
      pkgs.stdenv.cc.cc  
      pkgs.openssl.dev  
    ];  
  };  
}
```

Let's make it more realistic...

configuration.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.rustup  
    pkgs.gcc  
    pkgs.pkg-config  
    pkgs.openssl  
  ];  
  
  environment.sessionVariables = {  
    PKG_CONFIG_PATH = "${pkgs.openssl.dev}/lib/pkgconfig";  
    LD_LIBRARY_PATH = pkgs.lib.makeLibraryPath [  
      pkgs.stdenv.cc.cc  
      pkgs.openssl.dev  
    ];  
  };  
}
```

Pros and cons

- ⚙ Somewhat simple, but not simple to find or guess
- ⚙ “Works On My Machine.” (configuration.nix is your machine.)
- ⚙ Doesn't scale/compose well for many projects with many dependencies
- ⚙ Nix chatrooms are very unhelpful when you want to do this. Don't do this!



A quick buzzword collection (1/2)

- ❄️ **Nix**: A cross-platform package manager (`nix` and `nix-*` commands)
- ❄️ **Nix**: A domain-specific, lazy, functional, dyn. typed programming language
- ❄️ **Nixpkgs**: The largest, most up-to-date software distribution in the world:
 - ❄️ written in the Nix language,
 - ❄️ installed with the Nix package manager.
- ❄️ **NixOS**: A Linux-based operating system made with Nix and Nixpkgs

A quick buzzword collection (2/2)

- ⚙️ **Nix expression**: An expression in the Nix language
- ⚙️ **configuration.nix**:
 - ⚙️ NixOS is defined by a single Nix expression, often stored in this file
 - ⚙️ NixOS initially places this file in `/etc/nixos/configuration.nix`
 - ⚙️ Nix expressions allow `import` ing sub-expressions in other files
- ⚙️ **shell.nix, default.nix, flake.nix**:
 - ⚙️ Used by the Nix package manager, committed to a project repository
 - ⚙️ Useful for dev shells, build environments, and exporting build artifacts
 - ⚙️ Each of these contain a single Nix expression
- ⚙️ **Derivation**: A data structure that describes how to build a package. It takes inputs (dependencies), build steps, and outputs (build artifacts). A simple derivation can represent a single program, and a composite derivation can represent an entire operating system.

Entering shell.nix

shell.nix

```
let
  pkgs = import <nixpkgs> {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

Entering shell.nix

shell.nix

```
let
  pkgs = import <nixpkgs> {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

command-line

```
$ nix-shell
[nix-shell:~/hello-world] $ cargo run
...
  "ip": "91.198.204.39",
  "country": "DK",
  "city": "Copenhagen",
  ...
```



Entering shell.nix

shell.nix

```
let
  pkgs = import <nixpkgs> {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

command-line

```
$ nix-shell
[nix-shell:~/hello-world] $ cargo run
...
  "ip": "91.198.204.39",
  "country": "DK",
  "city": "Copenhagen",
  ...
```

Pros and cons

- ⚙ Automates parts of setting up the environment (gcc, environment variables)
- ⚙ Is located in your project directory
- ⚙ Lets you install anything from nixpkgs
- ⚙ Goodbye regular shell? 🤔 – I want my zsh aliases back!
- ⚙ What version is <nixpkgs>? 🤔 – reproducibility problems ahead



Goodbye regular shell? Ô_ô



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

Installing direnv in NixOS...

direnv.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.direnv  
  ];  
  
  programs.direnv = {  
    enable = true;  
    enableZshIntegration = true;  
    nix-direnv.enable = true;  
  };  
}
```

Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

Installing direnv in NixOS...

direnv.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.direnv  
  ];  
  
  programs.direnv = {  
    enable = true;  
    enableZshIntegration = true;  
    nix-direnv.enable = true;  
  };  
}
```

configuration.nix

```
{pkgs, ...}: {  
  imports = [  
    ./hardware-configuration.nix  
    ./vscode.nix  
    ./direnv.nix  
    ./zsh.nix  
  ];  
  
  # ...  
}
```

Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

Installing direnv in NixOS...

direnv.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.direnv  
  ];  
  
  programs.direnv = {  
    enable = true;  
    enableZshIntegration = true;  
    nix-direnv.enable = true;  
  };  
}
```

configuration.nix

```
{pkgs, ...}: {  
  imports = [  
    ./hardware-configuration.nix  
    ./vsodium.nix  
    ./direnv.nix  
    ./zsh.nix  
  ];  
  
  # ...  
}
```

Followed by...

```
$ sudo nixos-rebuild switch
```



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

Installing direnv in NixOS...

direnv.nix

```
{pkgs, ...}: {  
  environment.systemPackages = [  
    pkgs.direnv  
  ];  
  
  programs.direnv = {  
    enable = true;  
    enableZshIntegration = true;  
    nix-direnv.enable = true;  
  };  
}
```

Followed by...

```
$ sudo nixos-rebuild switch
```

configuration.nix

```
{pkgs, ...}: {  
  imports = [  
    ./hardware-configuration.nix  
    ./vscode.nix  
    ./direnv.nix  
    ./zsh.nix  
  ];  
  
  # ...  
}
```

Or, if you're not on NixOS...

```
$ <apt/brew/...> install direnv  
$ echo 'eval "$(direnv hook zsh)"' >> ~/.zshrc
```



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

And now, for enabling direnv...

```
[nix-shell:~/hello-world] $ ^D
$ echo 'use nix' > .envrc
$ direnv allow
```



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

And now, for enabling direnv...

```
[nix-shell:~/hello-world] $ ^D
$ echo 'use nix' > .envrc
$ direnv allow

direnv: loading ~/hello-world/.envrc
direnv: using nix
direnv: nix-direnv: Using cached dev shell
direnv: export +AR +AS +CC +CONFIG_SHELL +CXX +HOST_PATH +IN_NIX_SHELL +LD +NIX_BINTOOLS
+NIX_BINTOOLS_WRAPPER_TARGET_HOST_x86_64_unknown_linux_gnu +NIX_BUILD_CORES +NIX_CC
+NIX_CC_WRAPPER_TARGET_HOST_x86_64_unknown_linux_gnu +NIX_CFLAGS_COMPILE
+NIX_ENFORCE_NO_NATIVE +NIX_HARDENING_ENABLE +NIX_LDFLAGS ...
```



Goodbye regular shell? Ô_ô

Improving on shell.nix with direnv + nix-direnv

And now, for enabling direnv...

```
[nix-shell:~/hello-world] $ ^D
$ echo 'use nix' > .envrc
$ direnv allow

direnv: loading ~/hello-world/.envrc
direnv: using nix
direnv: nix-direnv: Using cached dev shell
direnv: export +AR +AS +CC +CONFIG_SHELL +CXX +HOST_PATH +IN_NIX_SHELL +LD +NIX_BINTOOLS
+NIX_BINTOOLS_WRAPPER_TARGET_HOST_x86_64_unknown_linux_gnu +NIX_BUILD_CORES +NIX_CC
+NIX_CC_WRAPPER_TARGET_HOST_x86_64_unknown_linux_gnu +NIX_CFLAGS_COMPILE
+NIX_ENFORCE_NO_NATIVE +NIX_HARDENING_ENABLE +NIX_LDFLAGS ...

$ cargo run
...
  "ip": "91.198.204.39",
  "country": "DK",
  "city": "Copenhagen",
  ...
```

Welcome back regular shell! ^_^



What version is `<nixpkgs>`? $\hat{_}\hat{_}$

What version is <nixpkgs>? ô_Ô

Addressing reproducibility: shell.nix doesn't have a lock-file...

What version is <nixpkgs>? ô_Ô

Addressing reproducibility: shell.nix doesn't have a lock-file...
shell.nix

```
let
  pkgs = import <nixpkgs> {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

What version is <nixpkgs>? ô_Ô

Addressing reproducibility: shell.nix doesn't have a lock-file...

shell.nix

```
let
  pkgs = import <nixpkgs> {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

What this syntax means

- ❄ <...> resembles #include <stdio.h>
- ❄ Looks in \$NIX_PATH (“Nix search path”)
- ❄ Looks at current Nix *channels* installed
- ❄ Alternative: use builtins.fetchTarball



What version is <nixpkgs>? ô_Ô

Addressing reproducibility: shell.nix doesn't have a lock-file...

shell.nix

```
let
  # nixpkgs = "https://github.com/NixOS/nixpkgs/archive/refs/tags/24.05.tar.gz";
  # nixpkgs = "https://github.com/NixOS/nixpkgs/archive/refs/tags/24.11-pre.tar.gz";
  nixpkgs = "https://github.com/NixOS/nixpkgs/archive/nixos-unstable.tar.gz";
  pkgs = import (builtins.fetchTarball nixpkgs) {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```



What version is <nixpkgs>? ô_Ô

Addressing reproducibility: shell.nix doesn't have a lock-file...

shell.nix

```
let
  # nixpkgs = "https://github.com/NixOS/nixpkgs/archive/refs/tags/24.05.tar.gz";
  # nixpkgs = "https://github.com/NixOS/nixpkgs/archive/refs/tags/24.11-pre.tar.gz";
  nixpkgs = "https://github.com/NixOS/nixpkgs/archive/nixos-unstable.tar.gz";
  pkgs = import (builtins.fetchTarball nixpkgs) {};
in
  pkgs.mkShell {
    packages = with pkgs; [
      rustup
      pkg-config
      openssl
    ];
  }
```

Welcome back reproducibility! ^_^



GitHub Actions + Nix

GitHub Actions + Nix

default.nix

```
{pkgs ? import <nixpkgs> {}}:
pkgs.rustPlatform.buildRustPackage {
  pname = "hello-world";
  version = "0.1.0";
  src = ./.;
  cargoHash = "sha256-fCWxZbZRVBKcr27Z8Pz+mN...";
  nativeBuildInputs = with pkgs; [
    rustc
    cargo
    pkg-config
  ];
  buildInputs = with pkgs; [
    openssl
  ];
  buildPhase = ''cargo build --release'';
  checkPhase = ''
    cargo fmt --all -- --check
    cargo build --all-targets
    cargo clippy --all-targets -- -D warnings
    cargo test --all-targets
  '';
  installPhase = ''
    mkdir -p $out/bin
    cp target/release/hello-world $out/bin/
  '';
}
```

GitHub Actions + Nix

default.nix

```
{pkgs ? import <nixpkgs> {}}:
pkgs.rustPlatform.buildRustPackage {
  pname = "hello-world";
  version = "0.1.0";
  src = ./.;
  cargoHash = "sha256-fCWxzBZRVBKcr27Z8Pz+mN...";
  nativeBuildInputs = with pkgs; [
    rustc
    cargo
    pkg-config
  ];
  buildInputs = with pkgs; [
    openssl
  ];
  buildPhase = ''cargo build --release'';
  checkPhase = ''
    cargo fmt --all -- --check
    cargo build --all-targets
    cargo clippy --all-targets -- -D warnings
    cargo test --all-targets
  '';
  installPhase = ''
    mkdir -p $out/bin
    cp target/release/hello-world $out/bin/
  '';
}
```

.github/workflows/rust-nix.yaml

```
name: Rust CI with Nix
on:
  workflow_dispatch:
  pull_request:
  push:
jobs:
  nix-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          nix_path: nixpkgs=channel:nixos-unstable
      - run: nix-build --arg doCheck true
```

GitHub Actions + Nix

default.nix

```
{pkgs ? import <nixpkgs> {}}:
pkgs.rustPlatform.buildRustPackage {
  pname = "hello-world";
  version = "0.1.0";
  src = ./.;
  cargoHash = "sha256-fCWxZbZRVBKcr27Z8Pz+mN...";
  nativeBuildInputs = with pkgs; [
    rustc
    cargo
    pkg-config
  ];
  buildInputs = with pkgs; [
    openssl
  ];
  buildPhase = ''cargo build --release'';
  checkPhase = ''
    cargo fmt --all -- --check
    cargo build --all-targets
    cargo clippy --all-targets -- -D warnings
    cargo test --all-targets
  '';
  installPhase = ''
    mkdir -p $out/bin
    cp target/release/hello-world $out/bin/
  '';
}
```

.github/workflows/rust-nix.yaml

```
name: Rust CI with Nix
on:
  workflow_dispatch:
  pull_request:
  push:
jobs:
  nix-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          nix_path: nixpkgs=channel:nixos-unstable
      - run: nix-build --arg doCheck true
```

Pros and cons

- ⚙️ You don't also need a shell.nix
- ⚙️ Automatically handles cargo-vendor
- ⚙️ Simple to maintain, so-so to discover
- ⚙️ Some redundancy from Cargo.toml
- ⚙️ Cargo workspace $\Rightarrow$ very boilerplate

A small digression about *vendoring*

Why complicate things with default.nix? shell.nix was working fine...

❄ The GitHub Action could just have used `nix-shell --run cargo ...`

A small digression about *vendoring*

Why complicate things with `default.nix`? `shell.nix` was working fine...

- ❄ The GitHub Action could just have used `nix-shell --run cargo ...`

Because of reproducible builds

- ❄ We assume unconditional internet access during build (to crates.io)
- ❄ `cargo-chef`: Speeds up Rust Docker builds using Docker layer caching
- ❄ `cargo-vendor`: Downloads all dependencies for caching or version control
- ❄ `pkgs.rustPlatform.buildRustPackage` calls `cargo-vendor` under the hood:
 - ❄ downloads and compiles them using a so-called *fixed-output derivation*
 - ❄ makes the current package depend on that
 - ❄ caches the result until the lockfile is bumped
 - ❄ avoids manual interaction with vendored crates

Now that I've sold you on some complexity...

┌┐ ^ \ (° □ °) / ^ ┐┐

Let's get more realistic: Cargo Workspaces, cross-compilation

More declarative alternatives to `rustup`: overlays

-  **rust-overlay**: select particular Rust version and components
-  **fenix**: also bundles nightly rust-analyzer and VSCode plugin

Packaging Rust projects with Nix

-  **buildRustPackage**: requires a `cargoHash = "sha256-..."`, boilerplate
-  **cargo2nix**, **erate2nix**: both generate a `Cargo.nix` file (causes git noise)
-  **dream2nix**: common interface across language toolchains, didn't try it
-  **naersk**: reduces boilerplate, handles `Cargo.lock`, incremental builds, caching
-  **crane**: improves on `naersk`: more granular build steps, better build caching, better cross-compilation support, better support for vendored dependencies

Flake helper libraries

-  **flake-utils**: avoids duplication for multiple architectures, very common
-  **flake-parts**: similar, but also well-documented, opinionated, and versatile

Example: A Cargo workspace for: kagi-api, kagi-cli

What are overlays? Specifically, rust-overlay

- 🦀 An overlay { modifies, adds, overrides } packages in nixpkgs
- 🦀 So: rust-overlay overrides the Rust toolchain in nixpkgs

flake.nix (Cargo workspace for: kagi-api, kagi-cli)

Some practical examples of flake.nix

-  kagi-api
-  web-sys-turtles
-  tauri-leptos-example



Setting up VSCode on NixOS

~_ (ツ) _/~

An explanation of the newer alternatives to nix-shell <https://blog.ysndr.de/posts/guides/2021-12-01-nix-shells/>: `nix shell`, `nix develop`, `nix run`